هند عبد الناصر محمد	sec 7	هشام مصطفى	sec 7
يارا ياسر ابو سريع	sec 7	عمر حماده	sec 5

Toxic Comment Classification

1. Introduction

This project focuses on detecting toxic comments using Natural Language Processing (NLP) and Machine Learning techniques. The system analyzes text comments and predicts whether they contain harmful or offensive language.
The project is based on a multi-label classification approach, which means that one comment can belong to more than one toxicity category at the same time.
Example:

- "You are stupid"
  → toxic = 1
  → insult = 1

The main goal of the project is to automatically identify toxic online comments and make online communication safer.

2. Dataset

The dataset used in this project is the Jigsaw Toxic Comment Classification dataset from Kaggle.
The dataset contains comments collected from online discussions and includes several toxicity labels:

- toxic
- severe_toxic
- obscene
- threat
- insult
- identity_hate

Each label has a value of:

- 1 → the comment belongs to this category
- 0 → the comment does not belong to this category

3. Data Preprocessing

Before training the model, the text data must be cleaned and prepared.
The preprocessing steps include:

1. Converting all text to lowercase.
2. Removing URLs and punctuation.
3. Removing stopwords such as "the" and "is".
4. Applying lemmatization to convert words into their base form.

Example:
"YOU are running!!!"
becomes:
"run"

These steps help improve the quality of the text data and reduce noise.

4. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to better understand the dataset.
The following analyses were applied:

- Checking missing values
- Checking duplicate values
- Analyzing class distribution
- Token length distribution
- Most frequent words
- N-grams analysis
- Word cloud visualization

The analysis showed that the dataset suffers from class imbalance because some classes contain many samples while others contain very few samples.

5. Feature Engineering

Machine learning models cannot understand raw text directly, so the text must be converted into numerical form.
TF-IDF (Term Frequency–Inverse Document Frequency) was used for feature extraction.
TF-IDF gives higher importance to important words and lower importance to common words.
Both:

- Unigrams
- Bigrams

were used to improve text representation.

6. Handling Class Imbalance

The dataset is imbalanced, meaning that some labels appear much more frequently than others.
To solve this issue, the project used:
class_weight='balanced'& threshold
This technique gives higher importance to minority classes during training.
It was chosen instead of SMOTE because TF-IDF produces sparse vectors and SMOTE is not ideal for sparse NLP data.

7. Model Training

Several machine learning models were trained and compared, including:

- Logistic Regression
- Support Vector Machine (LinearSVC)
- Decision Tree
- Random Forest
- AdaBoost

Since the task is multi-label classification, MultiOutputClassifier was used to allow the model to predict multiple labels for each comment.
Among all models, LinearSVC achieved the best performance because it works efficiently with high-dimensional sparse TF-IDF vectors.

8. Model Evaluation

The models were evaluated using several metrics:

- Accuracy
- F1-score

F1-score was considered the most important metric because the dataset is imbalanced.
The results showed that the SVM model achieved the highest overall performance.

9. LLM Integration

As a bonus feature, a Cloud Large Language Model (LLM) was integrated using Groq API.

The LLM does not perform classification itself. Instead, it explains the model's predictions in simple human-readable language.

Example:

Input comment:
"You are stupid"

Prediction:
toxic = 1
insult = 1

LLM explanation:
"The comment was classified as toxic because it contains insulting and offensive language directed toward another person."

This improves interpretability and user understanding.

10. User Interface

A simple interactive interface was built using Gradio.
The interface allows the user to:

- Enter a comment
- Receive toxicity predictions
- View confidence scores
- Read the AI-generated explanation

This makes the system easier to test and use interactively.

11. Conclusion

This project successfully applies NLP and machine learning techniques to detect toxic comments automatically. The project includes:

- Text preprocessing
- TF-IDF feature extraction
- Multi-label classification
- Class imbalance handling
- Model comparison
- LLM explanation generation
- Interactive Gradio interface

The final system can accurately classify toxic comments and explain the predictions in simple language, making it both effective and user-friendly.